LLM + RAG Mastery Project

Build a Research & Coding Assistant — an 8-Week Plan

This plan takes you from LLM fundamentals to a deployed, evaluated, production-shaped RAG + agent system. The end project is a coding and research assistant: point it at a codebase or a set of papers/docs, and it answers questions, cites sources, writes and runs code, and searches when it doesn't know something. Pitched for intermediate ML background (comfortable with Python and basic ML, new to LLMs/RAG specifically).

How to Use This Plan

- Each week has a goal, topics, a concrete deliverable, and resources. Don't move on until the deliverable works.
- Build one repo incrementally — each week adds a layer onto the same codebase, so by Week 8 it's one coherent project, not eight disconnected demos.
- Budget roughly 8-10 hrs/week. Weeks 3-5 are the heaviest (core RAG + agent logic).
- Use Claude or GPT-4 class models via API for the main pipeline; optionally run a local model (Ollama + Llama/Qwen) in parallel to compare cost/quality tradeoffs.

Architecture at a Glance

User query → API layer (FastAPI) → Agent Orchestrator (ReAct-style loop) → coordinates three things: (1) the LLM for reasoning/generation, (2) a Retriever backed by a vector DB + BM25 keyword index + reranker, (3) Tools for code execution, web search, and git access. Everything the retriever serves comes from an Ingestion Pipeline: load documents/code → chunk (AST-aware for code, semantic for text) → embed → store. Eval and logging wrap the whole loop, tracking answer quality, latency, and cost. See the diagram above for the visual version.

Weekly Roadmap

WEEK 1 LLM Foundations

Understand how LLMs actually work well enough to reason about failures, not just call an API.

Topics to master:

- Tokenization (BPE), context windows, and why token count = cost + latency
- Embeddings: what a vector represents, cosine similarity vs. dot product
- Prompt engineering: system/user/assistant roles, few-shot, chain-of-thought, structured output (JSON mode)
- Calling LLM APIs: Anthropic Claude API and/or OpenAI API; streaming responses
- Run a model locally with Ollama (e.g. Llama 3 or Qwen) to compare against hosted APIs
- Set up the project repo, virtual env, API keys, and a `.env` config pattern

Deliverable: A CLI script that takes a question, calls the LLM with a system prompt, and streams back an answer. Includes a token/cost counter.

Key resources: Anthropic prompt engineering docs, OpenAI API docs, 'Attention is All You Need' skim, Ollama quickstart

---

WEEK 2 RAG Basics

Get a naive but working retrieve-then-generate pipeline running end to end.

Topics to master:

- Document loaders: PDF, Markdown, and plain text ingestion
- Chunking strategies: fixed-size, recursive character splitting, overlap tradeoffs
- Embedding models: OpenAI/Cohere embeddings vs. open-source (e.g. bge, e5) via sentence-transformers
- Vector stores: Chroma or FAISS for local dev; when you'd reach for Qdrant/Pinecone in production
- Basic retrieval: top-k similarity search, injecting retrieved chunks into the prompt
- Why naive RAG breaks: lost-in-the-middle, chunk boundary issues, irrelevant retrievals

Deliverable: Ingest a folder of PDFs/docs into Chroma and answer questions over them with citations back to source chunks.

Key resources: LangChain or LlamaIndex RAG quickstart, Chroma docs, 'Lost in the Middle' paper (Liu et al.)

WEEK 3 Advanced RAG

Fix the failure modes from Week 2 — this is what separates a demo from something usable.

Topics to master:

- Hybrid search: combine dense (vector) + sparse (BM25) retrieval and merge results
- Reranking retrieved chunks with a cross-encoder (e.g. Cohere rerank or bge-reranker) before generation
- Query transformation: query rewriting, HyDE (hypothetical document embeddings), multi-query expansion
- Metadata filtering (by source, date, file type) and structured retrieval
- Context window management: how much retrieved content to actually feed the LLM, and in what order

Deliverable: Upgrade the Week 2 pipeline with hybrid search + reranking; measure retrieval precision before/after on a small test set of questions.

Key resources: 'Hybrid Search Explained' (Pinecone/Weaviate blogs), Cohere Rerank docs, HyDE paper

---

WEEK 4 Agents & Tool Use

Move from 'answer from retrieved text' to 'decide what to do, then do it.'

Topics to master:

- Function/tool calling with the Anthropic or OpenAI API — defining tool schemas
- The ReAct pattern: reason → act → observe → repeat
- Building a simple agent loop by hand (don't just import a black-box AgentExecutor — understand the loop first)
- Giving the agent tools: a retrieval tool, a code-execution sandbox tool, a web search tool
- Handling multi-step tasks, tool errors, and stopping conditions (avoiding infinite loops)

Deliverable: An agent that can decide whether to retrieve, search the web, or run code to answer a question — with visible reasoning traces.

Key resources: ReAct paper (Yao et al.), Anthropic tool use docs, LangGraph docs (optional, for structured agent graphs)

WEEK 5 Code-Aware Retrieval

Specialize the pipeline for the 'coding assistant' half of the project.

Topics to master:

- Parsing code with tree-sitter or Python's `ast` module for structure-aware chunking (functions/classes, not arbitrary line splits)
- Indexing an entire codebase: file-level + symbol-level embeddings
- Multi-file context assembly: pulling in a function plus its callers/imports when relevant
- Git integration: diffing, blame, and commit history as retrievable context
- Handling large repos: incremental re-indexing instead of full re-embeds on every change

Deliverable: Point the assistant at a real (medium-sized) open-source repo and have it correctly answer 'where is X implemented' and 'what calls Y' questions.

Key resources: tree-sitter docs, Sourcegraph/Cody engineering blog posts on code search, GitPython docs

---

WEEK 6 Evaluation & Guardrails

Stop trusting vibes. Measure whether the system is actually good, and catch it when it's wrong.

Topics to master:

- RAG evaluation metrics: faithfulness, answer relevance, context precision/recall (RAGAS framework)
- Building a small labeled eval set (20-50 Q&A pairs) specific to your indexed content
- Hallucination detection: citation-checking, asking the model to flag low-confidence answers
- Tracing and logging with Langfuse or LangSmith: see every retrieval + generation step
- Latency and cost tracking per query; setting budgets/alerts

Deliverable: An eval report (script + output) scoring the pipeline on your test set, with a before/after comparison against the naive Week 2 version.

Key resources: RAGAS docs, Langfuse docs, Anthropic's guide to reducing hallucinations

---

WEEK 7 Fine-Tuning & Optimization

Learn when fine-tuning actually helps (often it doesn't — RAG + prompting solves most problems) and how to make the system cheaper/faster.

Topics to master:

- When to fine-tune vs. when better retrieval/prompting is the right fix
- LoRA/QLoRA basics: parameter-efficient fine-tuning on a small open model
- Prompt caching (Anthropic/OpenAI) to cut repeated-context cost dramatically
- Model routing: cheap/fast model for simple queries, stronger model for complex reasoning
- Quantization basics for running models locally more efficiently

Deliverable: A short write-up + small experiment: fine-tune a small open model on a narrow task (e.g. code-comment generation) and compare it against prompting a base model.

Key resources: Hugging Face PEFT docs, Anthropic prompt caching docs, 'LoRA' paper (Hu et al.)

WEEK 8 Deployment

Ship it. Turn the project into something you (or anyone) can actually run and demo.

Topics to master:

- Wrapping the pipeline in a FastAPI backend with streaming responses
- A minimal UI: Streamlit for speed, or a small Next.js/React frontend for polish
- Containerizing with Docker; environment/secrets management
- Basic CI: lint + eval-set regression test on every change
- Monitoring in production: logging, error tracking, and re-running the Week 6 eval suite periodically

Deliverable: A deployed (or at least fully containerized and demo-able) research/coding assistant with a UI, README, and architecture diagram — your portfolio piece.

Key resources: FastAPI docs, Streamlit docs, Docker basics, 'Building LLM apps for production' (Chip Huyen's blog)

Suggested Repo Structure

Keep everything in one repo from Week 1 so the project reads as a coherent system by Week 8:

```
research-coding-assistant/  ingestion/      # loaders, chunkers, embedders  retrieval/     # vector store, BM25, reranker, hybrid search  agent/         # orchestrator, tool definitions, ReAct loop  tools/         # code_exec.py, web_search.py, git_tools.py  eval/          # RAGAS scripts, labeled test set, eval reports  api/           # FastAPI app  ui/            # Streamlit or frontend  Dockerfile, docker-compose.yml, README.md
```

Knowledge Check — By the End You Should Be Able To

- Explain, without hand-waving, why a RAG answer was wrong and which stage (chunking, retrieval, reranking, or generation) caused it
- Build a hybrid retrieval pipeline from scratch, not just call a framework's `.query()` method
- Write a tool-calling agent loop by hand and explain what LangChain/LlamaIndex are abstracting away
- Chunk and index a codebase in a structure-aware way, not just by character count
- Run a rigorous eval (RAGAS or equivalent) and quote real precision/recall/faithfulness numbers for your system
- Make an informed call on fine-tune vs. prompt/RAG for a given problem
- Deploy the whole thing behind an API with logging, cost tracking, and a basic CI eval gate